The Journal of Open Source Software

Catalyst: A Modular Framework for Graph Learning in Atomistic and Scientific Machine Learning

Miguel A. Tenorio[1], **Bamidele Aroboto**[1], **Lucille Sternberg**[1], **and James Chapman**[1]

1 Department of Mechanical Engineering, Boston University, Boston, MA 02215, USA

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 September 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

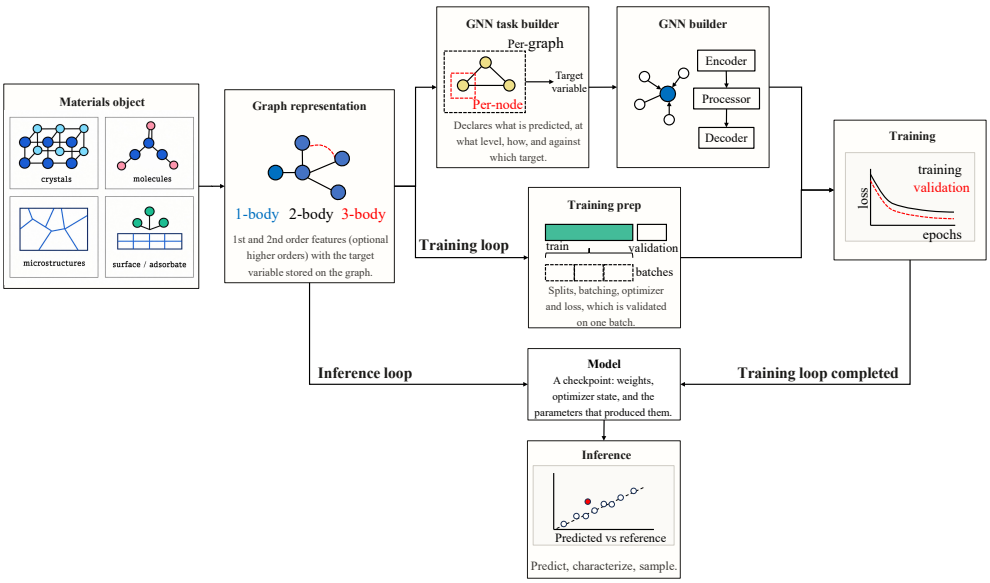


Figure 1: Catalyst provides a unified workflow that converts diverse materials representations, including crystals, molecules, microstructures, and surface/adsorbate systems, into graph-based inputs containing one-body, two-body, and optionally higher-order features together with the target property. The framework then defines the prediction task, constructs the encoder–processor–decoder GNN architecture, and manages dataset splitting, batching, optimization, and validation during training. Trained checkpoints retain the model and optimization state for subsequent inference, enabling property prediction, characterization, and sampling on new material structures.

Graph neural networks (GNNs) are increasingly used to learn relationships between structure and properties in materials, molecules, and other scientific systems. In atomistic applications, a structure is naturally represented as a graph: atoms form nodes, local interactions form edges, and higher-order geometric relationships can be represented using bond-angle or related graphs. In practice, however, a research workflow requires substantially more than a model architecture. Researchers must construct graphs, define targets with the correct tensor shapes and physical transformation behavior, train and checkpoint models, perform inference, and analyze predictions. Reimplementing these pieces for each project makes model comparison

The Journal of Open Source Software

difficult and creates opportunities for subtle inconsistencies.

Catalyst is an open-source Python framework that provides a common workflow for building, training, and analyzing graph-learning models for atomistic and generic scientific data. It is implemented on PyTorch and PyTorch Geometric (Fey & Lenssen, 2019; Paszke et al., 2019) and interoperates with the Atomic Simulation Environment (ASE) for atomistic structures (Larsen et al., 2017). Catalyst supports generic graphs, order-aware atomistic graphs containing atom, bond, and angular information, and equivariant graph models for geometry-dependent vector targets. A task abstraction defines whether a prediction is graph- or node-level and scalar, multiscalar, vector, or scalar-gradient based. This abstraction couples target semantics to training and validation while leaving model construction modular. Catalyst therefore targets researchers who want to develop, compare, and interpret graph-learning methods without rebuilding the surrounding scientific machine-learning workflow for each model.

Statement of Need

Scientific GNN development often combines several independent design choices: the graph representation, neural architecture, prediction target, loss accumulation, data-loading strategy, and analysis procedure. These choices are strongly coupled. A model that emits one scalar per graph cannot be trained correctly against a per-node target; three independent scalar properties are not equivalent to a three-component geometric vector; and a force-like vector target requires different transformation behavior from an invariant energy. Such mismatches may appear only during training, or worse, produce numerically valid but scientifically incorrect calculations.

Catalyst addresses this problem by treating the complete graph-learning workflow as a validated research object rather than treating training as an architecture-specific script. Its primary audience is researchers developing graph representations, scientific GNN architectures, surrogate models, and interpretable structure–property models. The same backend can be used for abstract graphs or atomistic structures and can support graph-level properties, local atomic quantities, vector fields, or multiple scalar observables. This is particularly useful for method-development studies in which the representation or model architecture is itself the experimental variable.

A second need is reproducibility across changing hardware and problem sizes. Atomistic datasets commonly contain graphs with highly variable numbers of atoms and edges. Catalyst provides checkpoint/restart support, CPU and GPU execution, optional mixed precision and compilation, node- or edge-budget batching, and distributed data-parallel training. These capabilities are exposed through a shared configuration system rather than project-specific training scripts.

State of the Field

Catalyst occupies a different layer of the scientific machine-learning ecosystem from general graph libraries and specialized interatomic-potential packages. PyTorch Geometric (Fey & Lenssen, 2019) and the Deep Graph Library (Wang et al., 2019) provide efficient graph data structures and message-passing primitives. Catalyst builds on this type of infrastructure but adds scientific task semantics, atomistic graph generation, validated training configuration, checkpoint/inference workflows, and analysis around a common public interface.

For atomistic learning, ALIGNN explicitly incorporates bond-angle information through line graphs (Choudhary & DeCost, 2021), while ALIGNN-d extends this idea to dihedral information and interpretable component contributions (Hsu et al., 2022). Equivariant frameworks such as e3nn provide general mathematical building blocks for Euclidean-equivariant networks (Geiger & Smidt, 2022), and packages such as NequIP and MACE focus on highly accurate equivariant

DRAFT

interatomic potentials (Batatia et al., 2022; Batzner et al., 2022). SchNetPack provides a broad atomistic machine-learning toolbox including training, equivariant models, and molecular dynamics (Schütt et al., 2023).

Catalyst is not intended to replace these packages. Its distinctive contribution is an architecture-agnostic workflow in which graph construction, encoder, processor, decoder, task definition, and training backend remain separable but mutually validated. It also spans generic graph problems and multiple atomistic representations under the same interface. Implementing this behavior directly in a low-level library such as PyTorch Geometric would impose domain-specific workflow assumptions on a general graph framework, whereas extending a single specialized potential package would retain assumptions tied to a narrower class of atomistic targets. Catalyst therefore acts as a research orchestration layer that can adopt ideas from specialized models while preserving a consistent experimental workflow.

Software Design

The central design of Catalyst is the decomposition

$$\mathcal{G} \xrightarrow{E} \mathbf{h} \xrightarrow{P} \mathbf{h}' \xrightarrow{D} \hat{\mathbf{y}},$$

where $\mathcal{G}$ is the input graph, E is an encoder, P is a processor or message-passing model, and D is a task-specific decoder. This separation allows researchers to change how structures are represented or processed without rewriting data loading, optimization, checkpointing, and inference.

Catalyst currently supports three broad graph families. Generic graphs allow arbitrary node and edge features and are useful for non-atomistic studies or representation-development experiments. Order-aware atomistic graphs encode atoms, pair interactions, and angular/line-graph relationships following the physical motivation of ALIGNN-type representations (Choudhary & DeCost, 2021). Equivariant atomistic graphs retain geometric information required for outputs such as forces or other vector fields.

A second design choice is to make the scientific task explicit. The `GNNTask` interface includes graph-scalar, graph-multiscalar, node-scalar, node-vector, graph-vector, and scalar-gradient tasks. The task specifies prediction level, output semantics, target field, and loss accumulation. Catalyst validates these contracts at multiple stages: configuration creation, task binding, model construction, and the final training preflight. This staged validation is intentional: a complete model does not need to exist when a configuration is created, but contradictions should be rejected as soon as the relevant components become available. The approach reduces silent errors caused by inconsistent tensor shapes or competing configuration values.

Performance features are opt-in rather than mandatory. Mixed precision, `torch.compile`, GPU prefetching, fused/foreach optimizers, TF32 controls, less-frequent validation, and CUDA/NCCL distributed training can be enabled without changing the scientific model definition. For datasets with highly variable graph sizes, mini-batches can be constrained by node or edge budgets instead of a fixed number of graphs. Conservative defaults preserve predictable numerical behavior and allow users to benchmark performance features for their own hardware and PyTorch versions.

The repository includes executable smoke workflows spanning generic graph regression, ALIGNN-style atomistic scalar learning, equivariant node-vector learning, multiscalar prediction, and end-to-end training/checkpoint/reload/inference. Continuous tests cover task contracts, atomistic graph construction and periodic neighbors, equivariance/invariance, staged parameter validation, and checkpoint behavior. The software is released under the MIT license, and the Python distribution is named `catalyst-gnn` to distinguish it from an unrelated package using the same general name.

DRAFT

Research Impact Statement

Catalyst consolidates recurring software requirements that emerged from graph-based materials research by the authors and collaborators, including topological graph descriptors for universal structure characterization (Chapman et al., 2022), angle- and dihedral-aware graph representations for spectroscopy (Hsu et al., 2022), local GNN measures of atomistic disorder (Chapman et al., 2023), and unsupervised graph learning of structural transitions (Aroboto et al., 2023). The current release turns those recurring research patterns into a tested public API. It provides end-to-end examples, an MIT-licensed repository, reproducible configuration and checkpointing, and automated tests across multiple supported Python versions. These features make Catalyst suitable both for internal method development and for distributing reproducible graph-learning workflows with publications. Its scientific value is strongest in research where the graph representation, target semantics, or model architecture must be systematically varied; for example, comparing invariant and equivariant models, separating global from atom-resolved predictions, or testing how explicit higher-order geometric information affects learned structure–property relationships.

Catalyst is currently being used within the Materials Informatics Laboratory as the graph-learning and training framework for ongoing atomistic machine-learning studies spanning structure–property prediction and the development of new graph representations. These active research workflows use the released Catalyst architecture for graph construction, model training, checkpointing, inference, and comparative model-development studies. Our prior SODAS/SODAS++ methodology is now integrated into Catalyst, and the Materials Informatics Laboratory routinely uses Catalyst in ongoing research on high-entropy alloys, catalytic reactions, and metal-organic framework characterization. Catalyst also provides the graph-learning foundation for active software development focused on latent-space analysis of atomistic trajectories and reactions and on compositional ordering in high-entropy alloys.

AI Usage Disclosure

OpenAI ChatGPT was used in drafting portions of the developer's manual and this manuscript, and to assist with brainstorming high-level workflow concepts, bug finding, and code optimization. GPT-5.6 Sol was used for the present manuscript drafting/conversion; model versions used in earlier development interactions were not consistently recorded. All AI-assisted material was reviewed and edited by the authors, who made the core software-design decisions and validated software behavior through source-code review and testing.

Acknowledgements

The authors thank contributors and users of the Materials Informatics Laboratory software ecosystem. The authors acknowledge support from the College of Engineering and Hariri Institute of Computing at Boston University.

References

Aroboto, B., Chen, S., Hsu, T., Wood, B. C., Jiao, Y., & Chapman, J. (2023). Universal and interpretable classification of atomistic structural transitions via unsupervised graph learning. *Applied Physics Letters*, *123*(9), 094103. https://doi.org/10.1063/5.0156682

Batatia, I., Kovács, D. P., Simm, G. N. C., Ortner, C., & Csányi, G. (2022). MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. *Advances in Neural Information Processing Systems*, *35*. https://doi.org/10.52202/068431-0830

DRAFT

Batzner, S., Musaelian, A., Sun, L., Geiger, M., Mailoa, J. P., Kornbluth, M., Molinari, N., Smidt, T. E., & Kozinsky, B. (2022). E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, *13*, 2453. https://doi.org/10.1038/s41467-022-29939-5

Chapman, J., Goldman, N., & Wood, B. C. (2022). Efficient and universal characterization of atomic structures through a topological graph order parameter. *Npj Computational Materials*, *8*, 37. https://doi.org/10.1038/s41524-022-00717-7

Chapman, J., Hsu, T., Chen, X., Heo, T. W., & Wood, B. C. (2023). Quantifying disorder one atom at a time using an interpretable graph neural network paradigm. *Nature Communications*, *14*, 4030. https://doi.org/10.1038/s41467-023-39755-0

Choudhary, K., & DeCost, B. (2021). Atomistic line graph neural network for improved materials property predictions. *Npj Computational Materials*, *7*, 185. https://doi.org/10.1038/s41524-021-00650-1

Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *International Conference on Learning Representations Workshop on Representation Learning on Graphs and Manifolds*. https://arxiv.org/abs/1903.02428

Geiger, M., & Smidt, T. (2022). e3nn: Euclidean neural networks. *arXiv Preprint arXiv:2207.09453*. https://doi.org/10.48550/arXiv.2207.09453

Hsu, T., Pham, T. A., Keilbart, N., Weitzner, S., Chapman, J., Xiao, P., Qiu, S. R., Chen, X., & Wood, B. C. (2022). Efficient and interpretable graph network representation for angle-dependent properties applied to optical spectroscopy. *Npj Computational Materials*, *8*, 151. https://doi.org/10.1038/s41524-022-00841-4

Larsen, A. H., Mortensen, J. J., Blomqvist, J., Castelli, I. E., Christensen, R., Dułak, M., Friis, J., Groves, M. N., Hammer, B., Hargus, C., Hermes, E. D., Jennings, P. C., Jensen, P. B., Kermode, J., Kitchin, J. R., Kolsbjerg, E. L., Kubal, J., Kaasbjerg, K., Lysgaard, S., … Jacobsen, K. W. (2017). The atomic simulation environment—a python library for working with atoms. *Journal of Physics: Condensed Matter*, *29*(27), 273002. https://doi.org/10.1088/1361-648X/aa680e

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., … Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, *32*. https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html

Schütt, K. T., Hessmann, S. S. P., Gebauer, N. W. A., Lederer, J., & Gastegger, M. (2023). SchNetPack 2.0: A neural network toolbox for atomistic machine learning. *The Journal of Chemical Physics*, *158*(14), 144801. https://doi.org/10.1063/5.0138367

Wang, M., Zheng, D., Ye, Z., Gan, Q., Li, M., Song, X., Zhou, J., Ma, C., Yu, L., Gai, Y., Xiao, T., He, T., Karypis, G., Li, J., & Zhang, Z. (2019). Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv Preprint arXiv:1909.01315*. https://doi.org/10.48550/arXiv.1909.01315

DRAFT